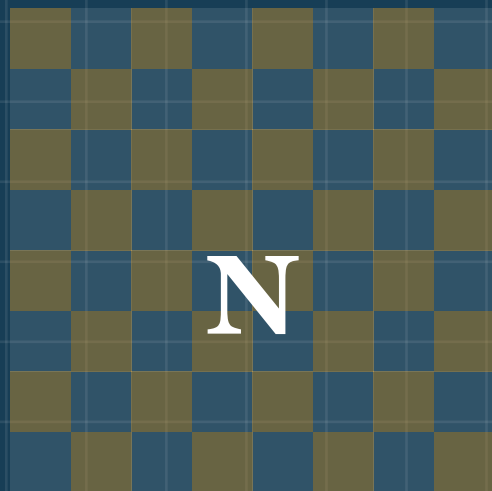


Anti-cheat Detective

A simple explanation of what was built,
how it works, and what it cannot prove



SECTION 01

The whole project in one minute

This page is the shortest possible explanation of the work.

What it is

A local research app that reads one chess game, asks Stockfish to evaluate every move, and shows simple measurements for White and Black.

What goes in?	A PGN file or pasted PGN text. PGN is the normal text format used to store a chess game.
What happens?	The app compares each played move with Stockfish's preferred move.
What comes out?	ACPL, median move loss, engine first-choice match rate, and the number of moves measured.
Does it say who cheated?	No. The current version intentionally does not give a cheating verdict or probability.
Where does it run?	Locally in Docker at http://localhost:8000 .

Most important rule

A person can play moves similar to an engine for many innocent reasons. Strong statistics can support an investigation, but they are not proof by themselves.

SECTION 02

Why the project exists

The goal is careful detection research, not an automatic accusation machine.

Chess engines can find extremely strong moves. A player who secretly receives engine help may make choices that look unusually engine-like. The research question is whether we can measure that pattern in a transparent and honest way.

The project started with five non-negotiable rules:

- Explain every number. The user should know where a measurement came from.
- Never let a language model invent or calculate raw statistics. It may receive only validated structured data in a later stage.
- Do not treat a controversial case as proof that the detector is correct.
- Use independently confirmed cases for future validation, and record the source of both the game and the outside evidence.
- Never advertise precision or recall as trustworthy when the sample is too small.

**Research
inspiration**

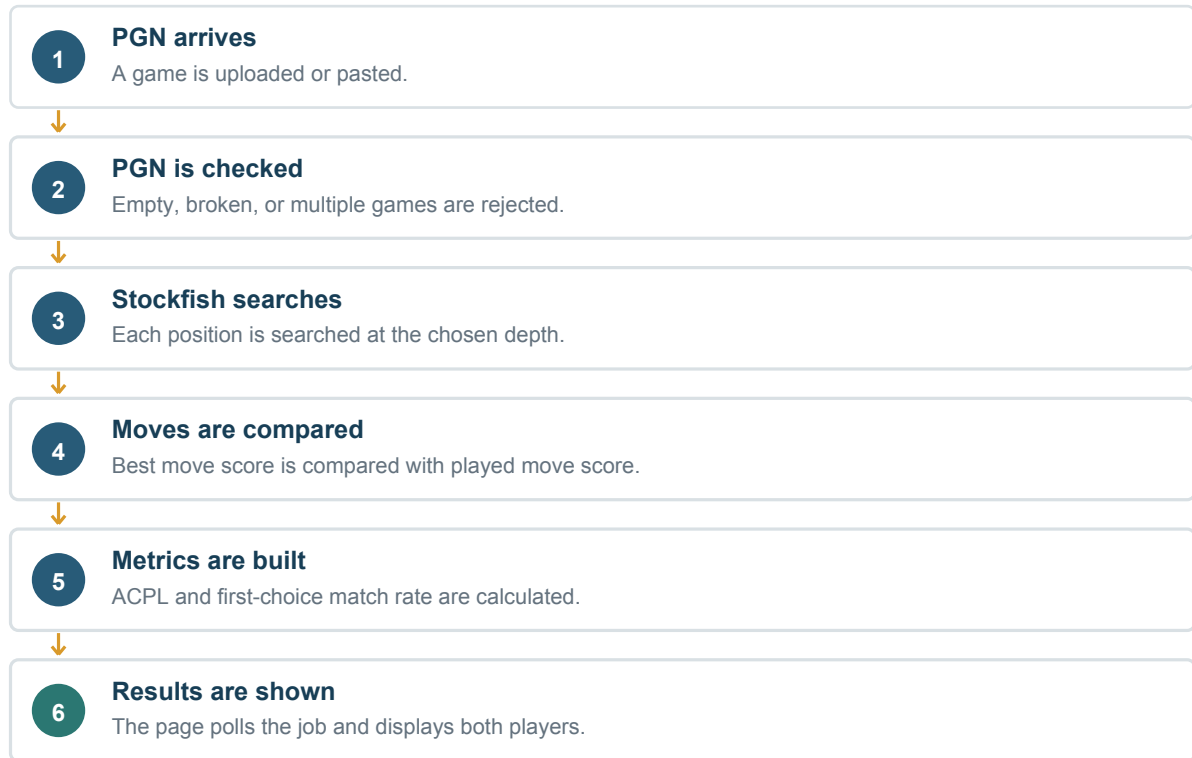
The long-term method is inspired by Kenneth Regan's statistical chess research. The current code implements only a transparent baseline, not the full Regan method.

The original plan has six stages: background research, data pipeline, advanced features, structured report writing, validation, and a final demo. The working prototype has completed the initial repository, the basic analysis engine, and the local user interface.

SECTION 03

What happens when you click Analyze

The browser hides the technical steps, but the process remains easy to audit.



Why two engine searches?

For each position, the app first lets Stockfish choose freely. Then it searches again while forcing the move that was actually played. This makes the comparison explicit.

SECTION 04

How one move is measured

A small made-up example shows the basic calculation.

Imagine it is White's turn. Stockfish says its best move gives White a score of +0.60 pawns. The move actually played gives White only +0.25 pawns.

Best move	+0.60 pawns = +60 centipawns
Played move	+0.25 pawns = +25 centipawns
Move loss	60 - 25 = 35 centipawns

All scores are converted to the point of view of the player who is moving. This matters because a positive score for White is a negative score for Black. Checkmate scores are converted to a large centipawn value so the calculations remain numeric.

Safety in the calculation

The code never records a negative loss. If two searches produce tiny ordering differences, the loss is limited to zero instead of suggesting the played move was better than the engine's own best move.

SECTION 05

The measurements in plain language

These are descriptions of move quality, not labels for human behavior.

ACPL	Average centipawn loss across a player's moves.	Lower usually means stronger or more accurate play.
Median loss	The middle move loss after losses are sorted.	Less affected by one very large mistake than the average.
Top-1 match rate	Share of moves equal to Stockfish's first choice.	Higher means more direct agreement with this engine at this depth.
Move count	Number of half-moves measured for that player.	A result based on 5 moves is much weaker than one based on many games.

The sample game used during testing had ten moves per player. White produced 2.9 ACPL and 90% first-choice matching; Black produced 3.7 ACPL and 60% matching. These numbers look very accurate because the sample is short and mostly opening theory. They do not suggest cheating.

**Do not compare
blindly**

ACPL depends on rating, time control, opening preparation, position difficulty, game phase, hardware, engine version, and search depth.

SECTION 06

The theory for future detection

Real statistical detection asks whether the full pattern is unexpectedly strong for the situation.

The planned approach starts with a normal-behavior model. It asks: for a player of this rating, in this kind of position and time control, how likely was each available move?

1**Build an expected baseline**

Use many ordinary games grouped by rating and time control.

2**Measure position difficulty**

A forced move should count less than a surprising engine-only move.

3**Look across many decisions**

A long repeated pattern matters more than one brilliant move.

4**Standardize the surprise**

Compare observed performance with the expected range and calculate uncertainty.

5**Test false positives**

Check strong clean players and controversial cases to learn when the method overreacts.

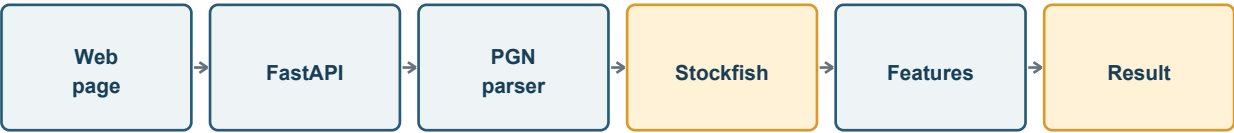
Current boundary

None of this future model is presented as finished. The current app stops at basic engine measurements and does not output a cheating score.

SECTION 07

How the software is organized

Each part has one clear responsibility.



Input and output stay on the local machine when the app runs locally.

web/index.html	The page for file upload, pasted PGN, progress, errors, and results.
api/app.py	Creates background jobs and exposes the health, create, and result endpoints.
engine/pgn.py	Reads exactly one valid non-empty chess game.
engine/stockfish.py	Starts Stockfish and compares every played move with the best move.
features/baseline.py	Calculates ACPL, median loss, match rate, and move count.
agent/schemas.py	Defines the validated numeric boundary for a future writing agent.
tests/	Checks parsing, metrics, engine behavior, API behavior, and the web page.

The API uses an in-memory job dictionary protected by a lock. This is enough for a local proof of concept. It is not yet designed for many users, multiple servers, or jobs that must survive a restart.

SECTION 08

How to use the finished app

The normal workflow no longer requires Swagger or command-line knowledge.

1**Start the app**

From the project folder, run: `docker compose up -d`

2**Open the page**

Visit `http://localhost:8000` in a browser.

3**Add a game**

Choose a .pgn file, paste PGN text, or press Load sample.

4**Analyze**

Press Analyze game. The page submits the job and checks progress automatically.

5**Read the evidence sheet**

Compare White and Black, but pay attention to move count and the limitations note.

6**Stop when finished**

Run: `docker compose down`

Privacy

When used at localhost, the PGN is sent only from the browser to the local Docker app on the same computer. The current code does not upload it to an outside service.

SECTION 09

What was done to make it trustworthy

Small engineering choices prevent silent errors and make the prototype reproducible.

One game per request	The app cannot silently analyze only the first game in a large pasted block.
Invalid PGN rejection	Broken or empty input fails before engine work begins.
Player point of view	White and Black scores are interpreted correctly for the person moving.
Recorded engine depth	A result shows the setting that influenced it.
Structured result schema	Unexpected or invented fields are rejected.
No accusation endpoint	The software cannot turn two baseline metrics into an unsupported verdict.
Docker image	Python, the application, and Stockfish run in a repeatable environment.

Verification completed on 19 July 2026:

- 15 automated tests passed.
- Total measured test coverage was 87%.
- The Stockfish wrapper reached 94% coverage using a fake engine for repeatable tests.
- The Ruff code-quality check passed.
- The Python wheel built successfully and included the web interface.
- Docker Compose rebuilt and started successfully.
- The complete browser flow was tested: load sample, analyze, poll, and show results.

SECTION 10

What the system cannot know

This is the most important page for responsible use.

The app sees chess moves. It does not see the person's thoughts, preparation, physical environment, communications, or intent. Therefore it cannot directly know whether assistance occurred.

Opening preparation	Known moves can match the engine for a long sequence.
Forced position	Only one or two sensible moves may exist.
Strong player	Elite players naturally choose engine-like moves more often.
Short game	A tiny sample can look extreme by chance.
Simple position	Accurate moves may be easy for many players.
Different engine settings	Depth, version, and hardware can change the preferred move.

What would support a real conclusion?

A careful statistical pattern across enough games, a well-calibrated comparison group, transparent uncertainty, and evidence outside the move statistics such as devices, communications, observation, or admission.

SECTION 11

What should be built next

The prototype is usable, but it is not yet a validated detector.

1	Create a licensed baseline-data manifest and download scripts.	Every game must have a known source and reproducible path.
2	Add a command-line analysis tool that writes JSON.	Batch research needs repeatable machine-readable outputs.
3	Add rating, time control, phase, complexity, and clock features.	Raw ACPL and match rate ignore major context.
4	Build clean controls and independently confirmed case sets.	Evaluation needs both false-positive and true-positive evidence.
5	Calibrate a simple statistical model before a complex model.	A transparent baseline is easier to audit.
6	Add validated report generation.	A writing agent should explain verified numbers, never create them.

Only after those steps should the project report precision, recall, or a suspiciousness score. Even then, the output should be treated as investigative support rather than a verdict.

SECTION 12

Glossary and references

A final quick reference for the main terms used in the project.

PGN	Portable Game Notation, a text format that stores chess moves and game details.
Stockfish	A very strong open-source chess engine.
Centipawn	One hundredth of a pawn, used as a small unit for chess evaluation.
ACPL	Average amount of evaluation lost per move.
Top-1 match	The played move is the engine's first choice.
UCI	A standard way for chess software to communicate with an engine.
API	A set of addresses that software uses to send requests and receive results.
Background job	Work that continues after the initial request returns.
False positive	A clean player or game is incorrectly treated as suspicious.

Methodology references planned for the final academic report

- Regan and Haworth - Intrinsic Chess Ratings (AAAI, 2011).
- Barnes and Hernandez-Castro - On the Limits of Engine Analysis for Cheating Detection (2015).
- Biswas and Regan - work on quantifying depth and complexity of chess thinking and knowledge.
- Lichess Irwin - architecture reference only; the project does not copy it directly.

Final takeaway

The project now has a working and understandable measurement tool. The honest next step is better data and calibration, not a stronger-looking accusation screen.